Creating and Sharing Genomic Annotation Modules with AI: The Just-DNA Ecosystem

Anonymous submission to EASRP 2026

Abstract

Genome analysis is an iterative workflow (ask, select data, score, annotate, inspect, and revise), and it is increasingly done inside AI coding assistants rather than in fixed graphical interfaces. But general-purpose language models are unreliable when used directly for genomics: they fabricate variant identifiers, misassign effect directions, and attach real PMIDs to the wrong paper. The Just-DNA ecosystem redirects these assistants from generating disposable code to producing structured, versioned data: *genomic annotation modules* where every variant is linked to its genotype and can carry its evidence, effect size, and a verbatim passage from the source paper. The output is an editable data artifact that a domain expert can review in a spreadsheet without specialized tools.

This paper presents the plugin ecosystem that makes this possible. The authoring plugin, **just-module-creator**, runs inside the AI coding assistants that bioinformaticians and developers already use daily, so module authoring requires no new infrastructure. The underlying toolchain validates authored files against a live schema, checks identifiers against reference databases, and compiles modules into a portable artifact. A shared registry lets independent authors publish, version, and exchange modules—the package-registry pattern applied to genomic annotation knowledge. Once installed, a module can be applied to a genome in about a minute; the annotation consumer, Just-DNA-Lite, is the subject of a companion paper [Anonymous, 2026a]. The software is open-source and available at `https://anonymous.4open.science/r/just-module-creator` (anonymized for review).

1 Introduction

Genomic annotation connects variant calls to the knowledge that gives them meaning: trait associations, clinical classifications, effect sizes, and the evidence behind each claim. This knowledge is scattered across papers, databases, and preprints, and it changes as new studies appear. Turning it into something a computational pipeline can use requires structured, validated records with unambiguous variant identifiers, genome-build coordinates, and traceable citations.

Researchers increasingly use AI coding assistants such as Claude Code and Codex for this kind of work. When asked to analyze variants, these tools typically generate a script: code that may fabricate identifiers [Shang et al., 2025], attach the wrong PMID to a real paper [Bhattacharyya et al., 2023], cite a source that does not support the claim [Gao et al., 2023], reverse an allele, or bypass established filters. Each session may produce slightly different code, and the result must be reviewed for correctness, efficiency, and security before it can be trusted. The output is a one-off answer tied to one session, not a reusable resource.

This paper presents the Just-DNA ecosystem, which gives these same assistants a different job. Instead of generating code, the assistant produces structured data: a *module* of editable CSV tables where each variant is linked to its studies, citations, effect sizes, and supporting passages. The input can be anything the assistant can read, from a single sentence describing a trait to a collection of papers or a summary from another model. When the module is enriched, and again when the registry publishes it, every variant's rsID is resolved against Ensembl's variation records, which carry dbSNP identifiers. Dedicated checking tools validate gene symbols against HGNC, trait identifiers against ontology services, and flag decisions that require domain expertise. Once the module is compiled to machine-readable Parquet files, Just-DNA-Lite joins it to a genome [Anonymous, 2026a]. Modules are versioned and shareable: they can be kept locally, rehearsed in a test registry, or published for others to install and use. The same files can also be written or edited entirely by hand.

This paper makes four contributions:

1. It describes the module contract shared by manual editors, AI-assisted authoring tools, versioned stores, and the genome-analysis consumer.
2. It presents `just-module-creator`, an authoring plugin that exposes the public capabilities of the ecosystem's format, compiler, enricher, and registry packages as typed Model Context Protocol (MCP) tools [Anthropic, 2024]. The plugin reads the installed schema instead of carrying a copy.
3. It defines how modules move between a local store, a test registry whose publications can be deleted, and the public catalog while preserving their provenance and the decisions that still require review.
4. It defines an evaluation protocol for variant recall, citation recall, and effect-direction agreement, and reports catalog statistics from the first eight published modules across three independent namespaces.

2 Related work

Genome annotation and catalogs. A variant call records what differs between a sample and a reference genome. Annotation adds information about that call, such as its predicted consequence, known clinical classification, reported trait association, or the conclusion assigned to a particular genotype. VEP, ANNOVAR, and OpenCRAVAT/OakVar perform this kind of lookup by joining a callset to established resources [McLaren et al., 2016, Wang et al., 2010, Pagel et al., 2020]. ClinVar and the GWAS Catalog publish records that can supply annotation content [Landrum et al., 2018, Sollis et al., 2023], HGMD derives variant–disease associations from the literature through manual review [Stenson et al., 2020], ClinGen provides expert curation within the ACMG/AMP framework [Rehm et al., 2015, Richards et al., 2015], and ClinPGx consolidates PharmGKB, CPIC, and PharmCAT into one pharmacogenomic resource [Whirl-Carrillo et al., 2012, ClinPGx, 2025, 2026]. CIViC crowdsources cancer variant interpretation, anchoring every evidence item to a source publication [Griffith et al., 2017]; its releases are database-wide rather than installable per-topic units, which makes it a source this ecosystem consumes rather than an alternative to it. The GA4GH Genomic Knowledge Standards define interoperable representations for variant annotations [Global Alliance for Genomics and Health, 2022]. The problem addressed here begins before the join: selected evidence must be turned into a reviewable, machine-checkable collection that can be distributed and applied consistently.

Table 1 narrows the comparison to systems adjacent to the authoring task: annotation engines, extension-based annotation platforms, expert-curated knowledgebases, structured literature extraction, unconstrained model drafting, and `just-module-creator`.

Biomedical text mining and variant extraction. Conventional named entity recognition systems such as tmVar3 and PubTator3 have improved gene and variant tagging in biomedical text [Wei et al., 2022, 2024]. LitVar 2 links literature paragraphs with variant records from external databases, accelerating knowledge retrieval [Allot et al., 2023]. However, as the PubMind authors note, these tools largely stop at tagging entities and do not recover relations such as a variant's pathogenicity in a particular disease or the experimental evidence behind a classification [Wang and Wang, 2026]. PubMind applies instruction-tuned LLMs to extract variant–disease–pathogenicity associations directly from over 41 million PubMed abstracts and 5.4 million full-text articles, producing a database of approximately 1.3 million unique variants with contextual annotations [Wang and Wang, 2026]. These systems demonstrate that LLM-based literature mining can recover structured variant knowledge at scale, but their output is

Table 1: Qualitative comparison of systems adjacent to annotation authoring. The rows have different primary tasks; the capability columns compare how their structured outputs are created, checked, and shared. Runtime annotation speed is outside this comparison and is covered in the companion platform paper [Anonymous, 2026a].

System	Kind / primary task	*Versioned modules*	*Schema validation*	*Provenance tracking*	*AI-assisted authoring*	*Shared catalog*
Ensembl VEP / ANNOVAR	annotation engine / consequence prediction against a callset	–	–	–	–	✓
OpenCRAVAT / OakVar	annotation platform / user-supplied annotation extensions	✓	✓	–	–	✓
CIViC	curated knowledgebase / expert-crowdsourced clinical interpretation	–	✓	✓	–	✓
PubMind-DB	text-mined database / LLM extraction of variant–disease–pathogenicity records	–	✓	✓	✓	✓
Raw LLM	general-purpose model / unconstrained drafting	–	–	–	✓	–
just-module-creator	authoring plugin / create, review, and publish installable modules	✓	✓	✓	✓	✓

Criteria. Versioned modules: user-installable or distributable annotation units with explicit versions. *Schema validation*: structured records are checked against a declared machine-readable schema. *Provenance tracking*: record- or module-level source links travel with the output. *AI-assisted authoring*: a language model participates directly in creating or revising the structured output. *Shared catalog*: outputs or extensions are discoverable through a shared service. A dash means the capability is not part of the system's core public workflow, not that no related function can be added around it.

a flat database rather than a versioned, schema-validated module that an annotation consumer can install and apply to a genome.

Factuality and tool use. Language models can produce fluent text that is unfaithful or nonsensical [Ji et al., 2023], including references with fabricated or wrong PubMed identifiers [Bhattacharyya et al., 2023]. A benchmark of genomic question answering found confident but wrong answers across models and tasks, among them gene name conversion and SNP locations [Shang et al., 2025]. Retrieval-augmented approaches and tool use reduce the need to improvise when a model can call a typed operation instead [Schick et al., 2023, Jin et al., 2024]. The Model Context Protocol provides a standard for connecting models to external tools [Anthropic, 2024]. Biomedical MCP services can search literature, ontologies, and variant databases. just-module-creator uses the same general approach, then carries the result into module files that the rest of the Just-DNA pipeline can inspect and reuse.

Agent orchestration. Multi-agent frameworks such as MetaGPT assign distinct roles to several model instances [Hong et al., 2024]. The plugin described here does not require its own orchestration runtime. Claude Code or Codex supplies the agent, while the plugin supplies the tools and the written workflow. This paper evaluates that authoring path, not a particular choice of language model or agent-team topology.

3 Method

3.1 The module artifact

Before it is compiled, a just-dna annotation module is an editable directory. It always contains `module_spec.yaml` and at least one recognized table. The table records the thing being annotated, such as a genotype, a diplotype, or a measured quantity. Evidence and machine-written sidecars (tables of records derived from reference sources) are stored separately. Compilation converts this human-readable directory into machine-readable Parquet files and a content-addressed `manifest.json` [Anonymous, 2026b]. The compiled artifact is what a consumer installs and joins to a genome.

The authored files are ordinary YAML, CSV, Markdown, and optional image files. A domain expert can open the tables in a spreadsheet to review and correct an AI draft without special tools. just-module-creator operates on this same directory rather than introducing a separate format for AI-produced content. The module describes how a genotype or measurement should be interpreted if a consumer encounters it. Just-DNA-Lite, or another compatible consumer, supplies the genome or measurement later.

The author does not need to supply this directory, or any CSV file, at the start of a session. Figure 1 shows how a conversation entry, which can range from a trait name to a collection of papers, moves through routing, scaffolding, and review to a compiled module. The `/create-module` router identifies the appropriate entry point and stage.

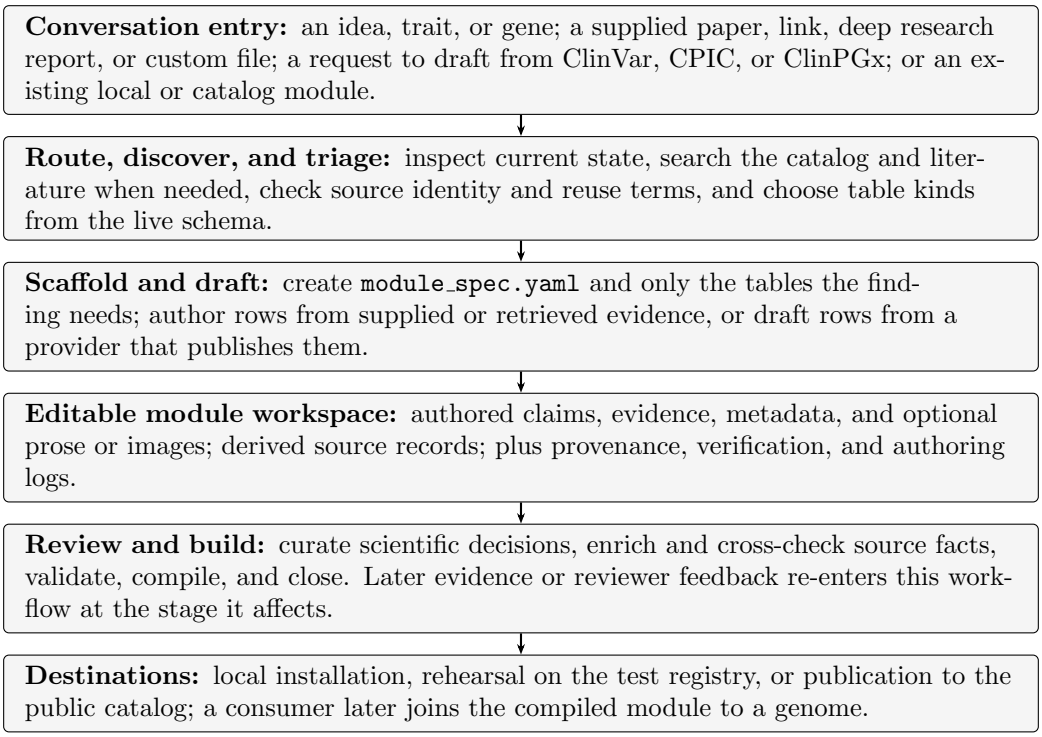


Figure 1: From conversation to a usable module. Module files are intermediate artifacts created by the workflow, not required user inputs. Authored claims remain distinct from derived source records inside the editable workspace.

Different claim types (variant associations, study evidence, pharmacogenomic diplotypes, copy-number ranges) belong to separate tables, each with its own schema. The plugin reads column definitions from the installed schema, so an author always sees the version accepted by the current compiler.

3.2 Architecture and lifecycle

Module authoring and sample analysis remain separate until Just-DNA-Lite joins a compiled module to a local genome. On the knowledge path, a person may edit the files directly or use an AI assistant to draft them. Both routes then pass through the same enrichment and compilation steps before the module enters a local or shared store. Figure 2 shows the knowledge and sample paths and the boundary between them.

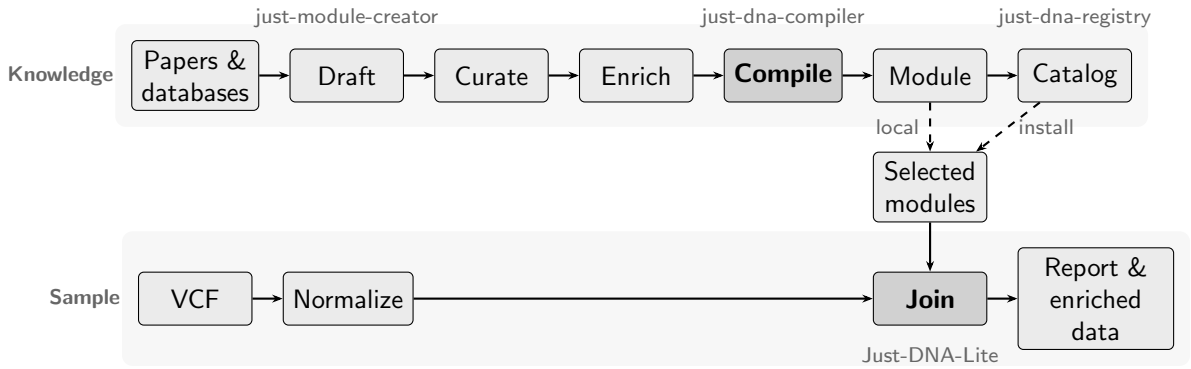


Figure 2: The two paths of the Just-DNA ecosystem.

On the sample path (Figure 2), Just-DNA-Lite converts VCF input to Parquet and joins it with compiled modules in a streaming Polars join, producing annotated reports and enriched data that the user can filter. With the ten modules of the companion paper's benchmark, the median time to annotate a whole-genome VCF ranged from 33 to 65 seconds across its four call sets on one workstation. The companion paper [Anonymous, 2026a] describes the sample path and polygenic risk score computation in detail.

Within `just-module-creator` the language model works on the knowledge path: it searches, reads and drafts. VCF filtering, module joins and polygenic scoring remain ordinary software steps with explicit inputs and repeatable code. One optional command crosses to the sample path: when the author asks, `/module-install-local` has the assistant run a module on genomes held by a local Just-DNA-Lite and read which variants each genome carries. The run goes through Just-DNA-Lite's own server, and the plugin's code itself still opens no genome. Those per-genome results then reach the assistant's model provider.

Table 2 lists each component's responsibility and boundary.

Table 2: Responsibilities across the Just-DNA ecosystem.

Component	Responsibility	Does not
`just-module-creator`	research and authoring workflow	process a genome
`just-dna-format`	schema and identity rules	access the network
`just-dna-compiler`	validate, compile, reverse, and sign	access the network
`just-dna-enricher`	resolve, draft, and cross-check	decide scientific meaning
`just-dna-registry`	publish, discover, and download modules	author module rows
Just-DNA-Lite	filter VCFs, join annotations, and produce reports	define module schema or compiler rules

3.3 Determinism, evidence, and responsibility

The compiler establishes structural validity and reproducibility: schema conformance, coordinate resolution, and a deterministic content signature [Anonymous, 2026b]. It does not establish

whether the authored claims agree with the literature: a false association in valid syntax compiles without error. Three properties keep the evidence chain auditable.

First, authored values and their checks are kept independent. A value filled from the source that will later check it makes the check compare that source with itself: a quote taken from an article's title is found in that article's own text. The design test is: *could this check have failed?* If not, it measured nothing (Section 4).

Second, disagreements with reference archives are preserved rather than silently resolved. ClinVar may lag a retraction; when the authored value should remain, `record_override` logs the difference and the reason.

Third, a check has three answers: it passed, it failed, or it never ran. The third is recorded as a null, and a null is not a *no*. The identifier check writes an attestation to `verification.json` so a reviewer can tell an empty report from one with no findings.

The assistant may locate a verbatim `provenance_quote` from retrieved full text and follow citations into supplementary tables to find per-variant statistics. A `curator` field records who located each quote (a name or model identifier) so a reviewer can direct scrutiny where it is most needed. Attribution does not transfer responsibility: the human author holds accountability regardless.

Curation follows drafting because a drafted row may still carry decisions the source cannot make (genotype, weight interpretation, conclusion wording), and these are surfaced for domain experts rather than resolved silently.

3.4 Local and shared module stores

A compiled module can reach a consumer through three routes (Figure 1): local installation into a Just-DNA-Lite checkout for testing with a VCF, publication to the test registry for a deletable rehearsal, or publication to the immutable public catalog. The registry runs its own enrichment and strict compilation on publish, so the stored artifact is the one the server produced, not a locally claimed digest. Inclusion in either catalog distributes a module; it is not scientific review, clinical validation, or endorsement.

3.5 Evaluation dimensions

The plugin's scorer evaluates variant recall over rsID–genotype pairs, citation recall (whether the run cites the papers the reference cites), and effect-direction agreement. Checking that a PMID names the intended paper is not yet implemented in the scorer. The reference standard is a human adjudication: a bioinformatician merged two earlier runs of the same prompt on the same paper and left the weights blank. Scoring modules against a module the same system authored would measure agreement with itself; the adjudication breaks that circle only in part, because every reference row was first drafted by the system.

Recall is reported at two grains, because recovering the right variant with wrong genotypes and recovering half the variants exactly are distinct failures that partial credit averages away. *Decoy rate* replaces precision: a variant absent from one curation may still be correct, whereas a decoy is one an expert designated non-associated.

Table 3: Three runs of one prompt scored against an expert-adjudicated reference module (a merge of two earlier runs), plugin version fixed.

Run	rsID recall	Pair recall	Decoy rate	Citation recall	Direction
1	1.00	0.67	0.00	1.00	1.00
2	1.00	1.00	0.00	1.00	1.00
3	1.00	1.00	0.00	1.00	1.00

Table 3 reports three runs of one prompt on one paper. All three recovered the variant the paper supports, cited both it and the earlier study it replicates, agreed on effect direction, and introduced no decoy. Two matched the reference on every genotype row, including a homozygous row that asserts nothing because neither cohort observed a carrier. All three declined to propagate the seed paper's description of the variant as stop-gained, which the study it cites classifies as intronic at the same coordinate.

The one divergence is a significance verdict at $p \approx 0.07$, read as *suggestive* by two runs and *not significant* by the third, which an aggregate score would have hidden. Agreement also measures the guidance and not only the runs: the validator names the missing genotype and a written rule states that a zero weight is a claim where a blank is not, so convergence shows the workflow is prescriptive, not that its output is confirmed.

A module can also be read without a reference curation, by checking what its cells claim against what they withhold. That reading corrected a prediction of ours: we expected a paper reporting no association test to produce no variant rows, yet one run produced sixty. Each recorded the observed allele but withheld effect direction and statistical significance. The blanks are honest: with no association test, there was nothing to report. Row count is therefore not part of the score; whether the filled cells claim more than the source supports is.

4 Results

With the plugin active, the assistant calls typed tools (Appendix B) instead of generating code: enrichment adds coordinates and derived sidecars, compilation validates against the live schema, and the compiled artifact can be published to the registry and immediately applied to a genome. The checks described in Section 3.3 are meant to catch citation misattributions, identifier drift, and schema violations, but an audit exposed a blind spot. The audit read working copies of four catalog modules, authored under an earlier version of the workflow and not the versions published in Table 4. In them, all 3,668 `studies.csv` rows carried the article's title as their `provenance_quote`, a passage that always matches its own full text and evidences nothing. The quote check had never run on them.

The plugin can be installed in two commands, one adding its marketplace and one installing it (`/plugin marketplace add` and `/plugin install` in Claude Code, and the matching `codex plugin` commands in Codex). It needs uv and Python 3.13 or newer. Once installed, the assistant gains access to 60 typed MCP tools (Appendix B) and 20 skills—written workflow instructions that teach the assistant when to call which tool. In both Claude Code and Codex, skills marked as commands can be invoked directly by typing `/name` in the prompt.

4.1 Command menu

Seven of the 20 skills are user-invocable commands (Table 15), exposing the workflow through intentions rather than internal stages. `/create-module` is the entry point: it reads where the author stands (a trait, a handed bundle, a directory somebody else left) and routes to the stage that owns the work. The remaining thirteen skills are guides, which a router loads by naming their path when the assistant reaches the corresponding lifecycle stage. A guide is deliberately not matched from its own description, which keeps thirteen documents out of every session's prompt and leaves a guide that nothing links to unreachable rather than merely quiet.

4.2 Production catalog

On 31 August 2026, the public catalog held eight published modules across 19 versions and three independent namespaces. Table 4 summarizes the published modules.

Two of the three namespaces belong to authors outside the development team, and four of the eight modules are theirs. Module sizes range from 2 to 474 variants. Four versions of

Table 4: Modules published to the public catalog as of 31 August 2026. All modules target GRCh38 and were compiled with strict resolution.

Module	Variants	Studies	Genes	Versions
big_five_personality_snps	330	390	185	4
risk_impulsivity_snps	474	530	325	3
cognitive_intelligence	32	89	31	3
aggression_anger_snps	28	28	22	4
bodybuilding	13	13	13	1
placebo_response_research	3	4	3	1
placebo_response_claude	3	9	3	1
lactose_tolerance	2	8	1	2
Total	**885**			**19 versions**

`big_five_personality_snps` show that a published module can be revised and republished. Two modules on the same trait (`placebo_response_research` via Codex, `placebo_response_claude` via Claude Code) were created by the same author to compare the two assistants; both compiled against the same schema and are published side by side.

5 Discussion

Genomic data is becoming personally accessible, and people already use AI assistants—coding tools such as Claude Code and Codex, or plain chat applications such as ChatGPT—to interpret their results [e.g., Ma, 2026]. Restricting this is not viable, so the more effective response is to make the work happen in the open, where domain professionals can review what others publish and the tools enforce structure. A module is visible, reviewable, and correctable; a one-off chat session is none of these.

The safety concern is real and extends beyond AI. Candidate-gene associations with depression have failed to replicate in large samples [Border et al., 2019], and GWAS effect sizes are systematically overestimated at discovery and shrink in replication cohorts [Palmer and Pe'er, 2017]. In direct-to-consumer raw data sent for clinical confirmation, 40% of variants were false positives [Tandy-Connor et al., 2018], and misclassified variants have led to genetic misdiagnoses that later reclassification exposed [Manrai et al., 2016]. The ecosystem is therefore explicitly research-only (Appendix A): a module that faithfully reflects a study that later fails to replicate is not wrong on its own terms, but a user acting on it may be harmed all the same.

Through the registry, a module one author publishes can be installed by others. The plugin's MCP server uses the standard stdio and HTTP transports of the Model Context Protocol, so other agent platforms that support the protocol can call its tools. This work used only Claude Code and Codex. What travels through the registry is genomic-data-agnostic knowledge: a module carries no individual-level data.

Limitations. The module system launched in August 2026. We are developing mechanisms to involve genetic counselors and domain-expert agents in quality evaluation, and to communicate to citizen scientists that many modules reflect early-stage research rather than clinical-grade evidence. Every run scored here used one model through one host, so the results say nothing about model or host sensitivity. Free-text conclusions are not scored at all, and the assistant's judgment about whether a source supports a claim remains a reviewer's question. Coordinates are declared rather than translated, and the compiler is bound to GRCh38. The file `module_spec.yaml` names a `genome_build`; an omitted one is taken to be GRCh38, and a GRCh37 or T2T declaration is recorded while its positions are still resolved as GRCh38. Liftover belongs to the source or the consumer; the PGS Catalog [Lambert et al., 2021] already

distributes scoring files harmonized to GRCh37 and GRCh38 [PGS Catalog, 2026]. Runtime benchmarks are in the companion paper [Anonymous, 2026a].

6 Conclusion

Annotation knowledge produced in a chat session is lost when the session ends. In the Just-DNA ecosystem, an AI assistant produces a versioned, schema-validated module instead of a script, so the work survives review, correction, and reuse by someone else. What makes that more than a file format is what a module keeps separate: authored claims stay distinct from derived records, a check that could not run says so, and each row of a published module has a place for the evidence behind it. Eight modules from three namespaces, half of them from authors outside the development team, show the path is functional today.

The software is open-source and intended for research use only. It installs in two commands in Claude Code or Codex, and a module drafted with it can be reviewed in a spreadsheet and published for others to install.

7 Code and data availability

- **just-dna-format, just-dna-compiler, and just-dna-enricher**: `https://anonymous.4open.science/r/just-dna-compiler` (schema, reference compiler, and enricher).
- **just-module-creator**: `https://anonymous.4open.science/r/just-module-creator` (MCP server, Claude Code and Codex plugin manifests, skills; MIT).
- **just-dna-registry**: `https://anonymous.4open.science/r/just-dna-registry` (registry service and reference client).
- **Just-DNA-Lite**: `https://anonymous.4open.science/r/just-dna-lite` (local VCF processing, annotation, and reporting; described in the companion paper [Anonymous, 2026a]).
- The public catalog and the test registry are live at URLs provided in the anonymized repositories.

References

Alexis Allot, Chih-Hsuan Wei, Lon Phan, Timothy Hefferon, Melissa Landrum, Heidi L. Rehm, and Zhiyong Lu. Tracking genetic variants in the biomedical literature using LitVar 2.0. *Nature Genetics*, 55:901–903, 2023. doi: 10.1038/s41588-023-01414-x.

Anonymous. Just-DNA-Lite: a local-first, open-source platform for personal genome annotation and polygenic risk scoring, 2026a. Companion platform paper (in preparation; citation anonymized for review).

Anonymous. just-dna-format: schema, compiler, and enricher for just-dna annotation modules. `https://anonymous.4open.science/r/just-dna-compiler`, 2026b. Software. Packages `just-dna-format`, `just-dna-compiler`, `just-dna-enricher`.

Anthropic. Model context protocol. `https://modelcontextprotocol.io`, 2024. Specification and documentation.

Mehul Bhattacharyya, Valerie M. Miller, Debjani Bhattacharyya, and Larry E. Miller. High rates of fabricated and inaccurate references in ChatGPT-generated medical content. *Cureus*, 15(5):e39238, 2023. doi: 10.7759/cureus.39238. PMID 37337480.

Richard Border, Emma C. Johnson, Luke M. Evans, Andrew Smolen, Noah Berley, Patrick F. Sullivan, and Matthew C. Keller. No support for historical candidate gene or candidate gene-by-interaction hypotheses for major depression across multiple large samples. *American Journal of Psychiatry*, 176(5):376–387, 2019. doi: 10.1176/appi.ajp.2018.18070881.

ClinPGx. Announcing ClinPGx. `https://blog.clinpgx.org/pharmgkb-is-now-clinpgx/`, July 2025. PharmGKB, CPIC and PharmCAT consolidated under a single pharmacogenomics resource.

ClinPGx. About ClinPGx. `https://www.clinpgx.org/about`, 2026. Accessed 2026-10-26.

Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6465–6488, 2023. doi: 10.18653/v1/2023.emnlp-main.398.

Global Alliance for Genomics and Health. GA4GH genomic knowledge standards. `https://www.ga4gh.org/work_stream/genomic-knowledge-standards-gks-work-stream/`, 2022. Interoperability standards for variant annotation and interpretation.

Malachi Griffith, Nicholas C. Spies, Kilannin Krysiak, et al. CIViC is a community knowledgebase for expert crowdsourcing the clinical interpretation of variants in cancer. *Nature Genetics*, 49(2):170–174, 2017. doi: 10.1038/ng.3774. PMID 28138153.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL `https://openreview.net/forum?id=VtmBAGCN7o`.

Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. doi: 10.1145/3571730.

Qiao Jin, Yifan Yang, Qingyu Chen, and Zhiyong Lu. GeneGPT: Augmenting large language models with domain tools for improved access to biomedical information. *Bioinformatics*, 40 (2):btae075, 2024. doi: 10.1093/bioinformatics/btae075.

Samuel A. Lambert, Laurent Gil, Simon Jupp, et al. The polygenic score catalog as an open database for reproducibility and systematic evaluation. *Nature Genetics*, 53(4):420–425, 2021. doi: 10.1038/s41588-021-00783-5. PMID 33692568.

Melissa J. Landrum, Jennifer M. Lee, Mark Benson, Garth R. Brown, Chen Chao, Shanmuga Chitipiralla, Baoshan Gu, Jennifer Hart, Douglas Hoffman, Wonhee Jang, Karen Karapetyan, Kenneth Katz, Chunlei Liu, Zenith Maddipatla, Adriana Malheiro, Kurt McDaniel, Michael Ovetsky, George Riley, George Zhou, J. Bradley Holmes, Brandi L. Kattman, and Donna R. Maglott. ClinVar: improving access to variant interpretations and supporting evidence. *Nucleic Acids Research*, 46(D1):D1062–D1067, 2018. doi: 10.1093/nar/gkx1153.

Jason Ma. An australian tech entrepreneur used AI to help create the first-ever bespoke cancer vaccine for a dog to treat his beloved pet Rosie. *Fortune*, March 2026. URL `https://fortune.com/2026/03/15/australian-tech-entrepreneur-ai-cancer-vaccine-dog-rosie-unsw-mrna/`. Accessed 2026-08-30.

Arjun K. Manrai, Birgit H. Funke, Heidi L. Rehm, Morten S. Olesen, Bradley A. Maron, Peter Szolovits, David M. Margulies, Joseph Loscalzo, and Isaac S. Kohane. Genetic misdiagnoses

and the potential for health disparities. *New England Journal of Medicine*, 375(7):655–665, 2016. doi: 10.1056/NEJMsa1507092.

William McLaren, Laurent Gil, Sarah E. Hunt, Harpreet Singh Riat, Graham R. S. Ritchie, Anja Thormann, Paul Flicek, and Fiona Cunningham. The Ensembl variant effect predictor. *Genome Biology*, 17:122, 2016. doi: 10.1186/s13059-016-0974-4.

Kymberleigh A. Pagel, Rick Kim, Kyle Moad, Ben Busby, Lily Zheng, Collin Tokheim, Michael Ryan, and Rachel Karchin. Integrated Informatics Analysis of Cancer-Related Variants. *JCO Clinical Cancer Informatics*, 4:310–317, 2020. doi: 10.1200/CCI.19.00132. OpenCRAVAT. OakVar is the maintained successor.

Cameron Palmer and Itsik Pe'er. Statistical correction of the winner's curse explains replication variability in quantitative trait genome-wide association studies. *PLoS Genetics*, 13(7): e1006916, 2017. doi: 10.1371/journal.pgen.1006916. PMID 28715421.

PGS Catalog. PGS Catalog downloads: Harmonized scoring files. `https://www.pgscatalog.org/downloads/`, 2026. Accessed 2026-10-26.

Heidi L. Rehm, Jonathan S. Berg, Lisa D. Brooks, Carlos D. Bustamante, James P. Evans, Melissa J. Landrum, David H. Ledbetter, Donna R. Maglott, Christa Lese Martin, Robert L. Nussbaum, Sharon E. Plon, Erin M. Ramos, Stephen T. Sherry, and Michael S. Watson. ClinGen — the clinical genome resource. *New England Journal of Medicine*, 372(23):2235–2242, 2015. doi: 10.1056/NEJMsr1406261.

Sue Richards, Nazneen Aziz, Sherri Bale, David Bick, Soma Das, Julie Gastier-Foster, Wayne W. Grody, Madhuri Hegde, Elaine Lyon, Elaine Spector, Karl Voelkerding, and Heidi L. Rehm. Standards and guidelines for the interpretation of sequence variants: a joint consensus recommendation of the American College of Medical Genetics and Genomics and the Association for Molecular Pathology. *Genetics in Medicine*, 17(5):405–424, 2015. doi: 10.1038/gim.2015.30.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Xinyi Shang, Xu Liao, Zhicheng Ji, and Wenpin Hou. Benchmarking large language models for genomic knowledge with GeneTuring. *Briefings in Bioinformatics*, 26(5):bbaf492, 2025. doi: 10.1093/bib/bbaf492. PMID 40984701.

Elliot Sollis, Abayomi Mosaku, Ala Abid, Annalisa Buniello, Maria Cerezo, Laurent Gil, Tudor Groza, Osman Güneş, Peggy Hall, James Hayhurst, Arwa Ibrahim, Yue Ji, Sajo John, Elizabeth Lewis, Jacqueline A. L. MacArthur, Aoife McMahon, David Osumi-Sutherland, Kalliope Panoutsopoulou, Zoë Pendlington, Santhi Ramachandran, Ray Stefancsik, Jonathan Stewart, Patricia Whetzel, Robert Wilson, Lucia Hindorff, Fiona Cunningham, Samuel A. Lambert, Michael Inouye, Helen Parkinson, and Laura W. Harris. The NHGRI-EBI GWAS catalog: knowledgebase and deposition resource. *Nucleic Acids Research*, 51(D1):D977–D985, 2023. doi: 10.1093/nar/gkac1010.

Peter D. Stenson, Matthew Mort, Edward V. Ball, Molly Chapman, Katy Evans, Luisa Azevedo, Matthew Hayden, Sally Heywood, David S. Millar, Andrew D. Phillips, and David N. Cooper. The Human Gene Mutation Database (HGMD): optimizing its use in a clinical diagnostic or research setting. *Human Genetics*, 139:1197–1207, 2020. doi: 10.1007/s00439-020-02199-3.

Stephany Tandy-Connor, Jenna Guiltinan, Kate Krempely, Holly LaDuca, Patrick Reineke, Stephanie Gutierrez, Phillip Gray, and Brigette Tippin Davis. False-positive results released by direct-to-consumer genetic tests highlight the importance of clinical confirmation testing for appropriate patient care. *Genetics in Medicine*, 20(12):1515–1521, 2018. doi: 10.1038/gim.2018.38.

Kai Wang, Mingyao Li, and Hakon Hakonarson. ANNOVAR: functional annotation of genetic variants from high-throughput sequencing data. *Nucleic Acids Research*, 38(16):e164, 2010. doi: 10.1093/nar/gkq603.

Peng Wang and Kai Wang. PubMind: literature-based genetic variant extraction and functional annotation using large language models. *Nature Communications*, 2026. doi: 10.1038/s41467-026-76834-4. Article in Press.

Chih-Hsuan Wei, Alexis Allot, Kevin Riehle, Aleksandar Milosavljevic, and Zhiyong Lu. tmVar 3.0: an improved variant concept recognition and normalization tool. *Bioinformatics*, 38(18): 4449–4451, 2022. doi: 10.1093/bioinformatics/btac537.

Chih-Hsuan Wei, Alexis Allot, Po-Ting Lai, Robert Leaman, Shubo Tian, Ling Luo, Qiao Jin, Zhizheng Wang, Qingyu Chen, and Zhiyong Lu. PubTator 3.0: an AI-powered literature resource for unlocking biomedical knowledge. *Nucleic Acids Research*, 52(W1):W540–W546, 2024. doi: 10.1093/nar/gkae235.

Michelle Whirl-Carrillo, Ellen M. McDonagh, J. M. Hebert, Li Gong, Katrin Sangkuhl, Caroline F. Thorn, Russ B. Altman, and Teri E. Klein. Pharmacogenomics knowledge for personalized medicine. *Clinical Pharmacology & Therapeutics*, 92(4):414–417, 2012. doi: 10.1038/clpt.2012.96.

Appendix

A Research use only

Annotation modules summarize published association findings. They are not clinical-grade evidence. The authoring plugin does not make individual-level predictions and does not open a genome. Just-DNA-Lite and `just-prs` process sample data for research and educational use; their reports and scores are not diagnoses or medical advice. Language that implies causation, such as "causes" or "guarantees," is outside the scope of a module written with this plugin.

B Plugin surface: tools and skills

A Claude Code or Codex plugin is a bundle of two kinds of asset. **MCP tools** are typed operations the assistant can call. Each has a name, typed inputs, and a structured return value. **Skills** are written instructions loaded into the assistant's context on demand; they teach the workflow that the tools serve, including when to call which tool and what to do with the result.

Tools (60)

The tools are organized into a core set and nine optional groups. By default all of them are listed; in an optional layered mode only the core set is listed and a session reveals the other groups through `toolbox`. Table 5 lists the 18 core and meta tools; Tables 6–14 list the nine groups. Registry writes require a token obtained through `authenticate`. Requests to each outside service pass through that service's pacing gate, and the literature search shares its NCBI gate with the enricher, so both spend one NCBI budget.

Table 5: Core tools (always listed) and the meta-tool.

Tool	Purpose
`list_tables`	List authorable table kinds and what each row represents
`describe_table`	Describe one table kind: every column, type, vocabulary, and pick-list
`table_requirements`	The three shapes of requiredness for a table kind
`describe_machine_table`	Describe a machine-written sidecar: columns, types, vocabulary
`get_template`	Get a CSV template (header only, or header plus stub rows)
`scaffold_module`	Create `module_spec.yaml` plus stub CSVs; never overwrites
`lint_rows`	Lint CSV text against a table kind (writes nothing)
`validate_module`	Pre-flight a spec directory (writes nothing)
`compile_module`	Compile a spec directory into Parquet plus `manifest.json`
`draft_from_clinvar`	Draft `variants.csv` and `studies.csv` from ClinVar
`enrich_module`	Resolve rsIDs to coordinates and mint VRS identifiers
`literature_search`	Find papers behind a row; confirm a PMID names the intended paper
`lookup_citation`	Check a PMID or DOI and read back which paper it names
`lookup_variant`	Look up one variant: loci, alleles, ClinVar calls, rsID currency
`record_override`	Log that an authored value deliberately outranks a source
`registry_register`	Create a registry account and mint its API key
`authenticate`	Provide a registry token to unlock write tools for this session
`toolbox`	List optional tool groups and reveal them to the session

Table 6: Evidence group: read a paper, its citations, and its supplementary tables.

Tool	Purpose
`lookup_open_access`	Where a paper may legally be read, and on what terms
`fetch_fulltext`	Retrieve a paper's text so the assistant can read and quote it
`paper_citations`	Papers that cite this one, or the papers it cites
`list_supplementary`	Inventory the supplementary files attached to a paper
`fetch_supplementary`	Download one supplementary file to the cache
`describe_supplementary`	Sheets in a downloaded workbook and their column names

Skills (20)

Skills are Markdown documents loaded into the assistant's context when needed. Seven are **commands** (Table 15), which the user invokes by typing `/name` in the assistant's prompt. The other thirteen are **guides** (Table 16), which a router loads by naming their path when the assistant reaches the corresponding stage.

Table 7: Identifiers group: check gene symbols and ontology CURIEs.

Tool	Purpose
`check_identifiers`	Check every gene symbol (HGNC) and trait CURIE (OLS4) in a spec
`lookup_identifier`	Check one gene symbol or trait CURIE

Table 8: Pharmacogenomics (PGx) group: draft from curated PGx sources.

Tool	Purpose
`draft_from_cpic`	Draft haplotypes, allele function, and diplotypes from CPIC
`draft_from_clinpgx`	Draft `pharm_variants.csv` from a ClinPGx snapshot

Table 9: Passes group: fill or re-derive machine-written sidecars.

Tool	Purpose
`enrich_facts`	Fill frequency, constraint, and dosage sidecars
`enrich_literature_pass`	Resolve every citation in `studies.csv` into `literature.csv`
`enrich_gwas_effects`	Record the GWAS Catalog's published effect sizes for the module's rsIDs
`refresh_sidecar`	Re-derive one sidecar against its source, keeping curated rows

Table 10: Review group: read a module back and surface decisions.

Tool	Purpose
`audit_module`	Ask what a curation pass would ask (offline, reads only)
`review_queue`	Rank the rows where somebody overruled a source
`review_logs`	Show what is in the module's logs (publishing them is permanent)
`study_facts`	Per-study facts the GWAS pass wrote into `gwas_effects.csv`

Table 11: Integrity group: compare, sign, verify, and reverse modules.

Tool	Purpose
`compare_modules`	What moved between two spec directories (offline)
`compare_to_published`	Is local data ahead of the catalog, and how?
`module_signature`	Content signature of raw authored data (no compile, no network)
`verify_artifact`	Re-hash every file in a compiled artifact and recompute the digest
`reverse_module`	Turn a compiled artifact back into an authored spec directory

Table 12: Catalog group: read the registry.

Tool	Purpose
`registry_search`	Search published modules (read-only, no token)
`registry_get_module`	Fetch one module's full record: card, readme, versions, manifest
`registry_health`	Is this registry instance up, and which instance is it?
`registry_is_published`	Has this authored data been published under any name?
`registry_namespace_available`	Check whether a namespace is legal and unclaimed
`registry_download`	Download and integrity-verify a published module version

Table 13: Publish group: write to the registry (token required).

Tool	Purpose
`registry_whoami`	Confirm the token is accepted and report the account
`registry_validate`	Validate a spec on the registry without publishing
`registry_check`	Full publish dry run: validation plus network checks
`registry_publish`	Publish a spec directory as a module version
`registry_amend_readme`	Replace a published version's readme (no version bump)
`registry_claim_namespace`	Claim a publishing namespace (irreversible on production)
`registry_yank`	Stop recommending a version (stays fetchable for pinned users)
`registry_unyank`	Restore a yanked version to listings
`registry_delete_version`	Hard-delete one rehearsed version from the test registry
`registry_delete_module`	Hard-delete all rehearsed versions from the test registry

Table 14: Closing group: finish and describe.

Tool	Purpose
`close_module`	Declare authoring finished, bound to its authored bytes
`describe_spec_file`	Describe `module_spec.yaml`: every key and block it carries
`authoring_reference`	The complete generated description of the authoring DSL as JSON

Table 15: User-invocable commands (7). Each is typed as `/name` in the assistant prompt.

Command	Purpose
`/create-module`	Router: begin or continue creating a module from any starting point
`/module-status`	Inspect a directory and identify the next decision
`/module-revise`	Begin another pass over an existing module
`/find-evidence`	Find and read evidence for a row
`/module-publish`	Rehearse on the test registry, then publish to the catalog
`/module-symptom`	Explain a message from the toolchain
`/module-install-local`	Install a compiled module into Just-DNA-Lite locally

Table 16: Guides (13). Each is loaded by a router or a preceding skill rather than invoked by the user directly.

Guide	Purpose
`module-101`	High-level map: what a module is, what the plugin can and cannot do
`module-start`	Stage 0–1: triage, licence, the spec
`module-draft`	Stage 2: draft rows from evidence or a provider
`module-curate`	Stage 3: curate scientific decisions
`module-enrich`	Stage 4: enrich with coordinates, identifiers, and sidecars
`module-check`	Stage 5: cross-check against reference databases
`module-compile`	Stage 6: validate and compile to Parquet
`module-close`	Stage 6b: declare authoring finished and bind to authored bytes
`module-refresh`	Re-run anything that already ran
`module-diff`	What moved, and the reading that means a source changed its answer
`module-tables`	Which table kind, where every file sits, the three on-disk shapes
`module-weights`	The weight column everyone fills and nobody declares
`module-consumer`	The annotation consumer's side of the seam